Listing 10.14 Searching multiple fields using `multi_match`

```
GET books/_search
{
  "_source": false,
  "query": {
    "multi_match": {
      "query": "Java",
      "fields": [
        "title",
        "synopsis",
        "tags"
      ]
    }
  },
  "highlight": {
    "fields": {
      "title": {},
      "tags": {}
    }
  }
}
```

Suppresses the source document from appearing in the results

Specifies the multi_match query

Defines the search criterion as the word "Java"

Searches across multiple fields provided in an array

Highlights the matches returned in the results

The `multi_match` query expects an array of fields along with the search criterion. We get the aggregated results by combining all the results for individual fields.

10.8.1 Best fields

You may wonder what the document's relevancy is when we search multiple fields. Fields where more words are matched score higher. If we search for "Java Collections" across multiple fields, a field (say, `synopsis`) where two words match is more relevant than a field with one (or no) matches. In this case, the document with this `synopsis` field is given a higher relevancy score.

Fields that match all the search criteria are called the *best fields*. In the previous example, assuming `synopsis` contains both words, *Java* and *Collections*, we can say that `synopsis` is the best field. Multi-match uses a `best_fields` type under the hood when running queries. This type is the default for `multi_match` queries. Of course, there are other fields, as we see shortly.

Let's rewrite the query from listing 10.14. This time, instead of letting Elasticsearch use the default setting (the `best_fields` type), we specifically override the `type` field.

Listing 10.15 Specifying the `best_fields` type explicitly

```
GET books/_search
{
  "_source": false,
  "query": {
    "multi_match": {
      "query": "Design Patterns",
      "type": "best_fields",
```

Sets the type of multi_match query to best_fields